

App Tareas - Documentacion tecnica

Fecha: 28 de abril de 2026

Stack: Node.js, Express 5, SQLite3, HTML, CSS, JavaScript vanilla

1. Arquitectura general

La aplicacion es un sistema CRUD de gestion de usuarios y tareas. El backend esta hecho con Node.js y Express. La base de datos es SQLite. El frontend es HTML y JavaScript puro, sin ningun framework.

Flujo de una peticion:

El navegador carga public/index.html y ejecuta public/app.js. Desde ahi se hacen llamadas fetch() a las rutas /users y /tasks del servidor. Express recibe la peticion y la pasa a routes/users.js o routes/tasks.js. El route handler ejecuta la consulta SQL usando la instancia db que exporta database.js. El resultado regresa como JSON al navegador, que actualiza el DOM.

Archivos principales:

server.js: punto de entrada de Express. Monta las rutas y sirve la carpeta public como archivos estaticos.
database.js: abre el archivo database.db y crea las tablas users y tasks si no existen. Exporta la instancia db.

routes/users.js: maneja POST /users para crear y GET /users para listar.

routes/tasks.js: maneja POST /tasks, GET /tasks (con JOIN para traer el nombre del usuario) y el nuevo PUT /tasks/:id.

public/index.html: toda la interfaz de usuario.

public/app.js: toda la logica del navegador.

public/styles.css: el diseño visual.

Modelo de datos:

```
users (id INTEGER PRIMARY KEY, name TEXT)
tasks (id INTEGER PRIMARY KEY, title TEXT, user_id INTEGER -> users.id)
```

2. Mejora 1 - Diseño CSS moderno

El archivo styles.css estaba vacio. Se escribio desde cero con un sistema de diseño limpio.

Reset y tipografia

Se aplica un reset para eliminar márgenes y paddings por defecto del navegador. La fuente usada es la del sistema operativo (system font stack) para que se vea nativa en cada plataforma.

```
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0; padding: 0;
}
body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  background: #eef2ff;
}
```

Layout en dos columnas con tarjetas

La pagina se divide en dos columnas usando CSS Grid. En pantallas pequenas (menos de 700px) las columnas se colapsan en una sola. Cada seccion esta dentro de una tarjeta blanca con bordes redondeados y sombra.

```

.grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1.5rem;
}
@media (max-width: 700px) {
  .grid { grid-template-columns: 1fr; }
}
.card {
  background: #fff;
  border-radius: 14px;
  padding: 1.5rem;
  box-shadow: 0 2px 16px rgba(79, 109, 245, 0.08);
}

```

Jerarquía de botones

El botón base tiene fondo azul y ancho completo. Hay variantes con clases adicionales para acciones secundarias. La clase más específica en CSS siempre gana, por lo que el orden en el archivo no afecta el resultado.

```

button          -> azul, ancho completo (acción principal)
button.btn-edit -> gris claro con borde azul (secundaria)
button.btn-save -> verde (confirmar cambios)
button.btn-cancel -> gris neutro (cancelar)

```

Estado de error visual

Cuando un campo está vacío al intentar enviar un formulario, JavaScript le agrega la clase `.error`. Eso cambia el borde a rojo. Cuando el usuario empieza a escribir, la clase se quita automáticamente.

```

input.error, select.error {
  border-color: #ef4444;
  background: #fff5f5;
}

```

3. Mejora 2 - Dropdown de usuarios

Antes habia un campo de texto donde el usuario debia escribir el ID numerico del usuario al que queria asignar la tarea. Eso es propenso a errores. Ahora hay un select que se llena automaticamente con los usuarios existentes.

Cambio en el HTML

Se reemplaza el input por un select:

```
<!-- Antes -->
<input id="userId" placeholder="ID Usuario">

<!-- Ahora -->
<select id="userSelect" oninput="this.classList.remove('error')">
  <option value="">Seleccionar usuario</option>
</select>
```

Como se puebla el dropdown en JavaScript

La funcion cargarUsuarios() hace fetch a /users, guarda los datos en la variable global allUsers y construye las opciones del select. Se llama al arrancar la pagina y cada vez que se crea un usuario nuevo.

```
async function cargarUsuarios() {
  const res = await fetch(API + '/users');
  allUsers = await res.json();

  const select = document.getElementById('userSelect');
  select.innerHTML =
    '<option value="">Seleccionar usuario</option>' +
    allUsers.map(u =>
      `<option value="${u.id}">#${u.id} - ${escapeHtml(u.name)}</option>`
    ).join('');
}
```

4. Mejora 3 - Validacion de campos vacios

Antes era posible crear usuarios o tareas con campos en blanco, insertando filas vacias en la base de datos. Ahora se valida todo antes de enviar el request al servidor.

Patron de validacion

Se usa una variable 'valid' que empieza en true. Si algun campo esta vacio, se le agrega la clase error y valid pasa a false. Al final, si valid es false, la funcion se detiene y no hace el fetch.

```
async function crearTarea() {
  const titleInput = document.getElementById('taskTitle');
  const selectInput = document.getElementById('userSelect');
  const title = titleInput.value.trim();
  const user_id = selectInput.value;

  let valid = true;
  if (!title) { titleInput.classList.add('error'); valid = false; }
  if (!user_id) { selectInput.classList.add('error'); valid = false; }
  if (!valid) return;

  await fetch(API + '/tasks', { ... });
}
```

Se usan trim() para ignorar espacios en blanco. Se revisan todos los campos antes del return para que el usuario vea todos los errores al mismo tiempo. La misma logica aplica en el formulario de edicion.

Atajo con la tecla Enter

Cada input tiene un atributo onkeydown que llama a la funcion correspondiente cuando se presiona Enter. Esto esta vinculado directamente al input para no interferir con otros formularios.

```
<input id="userName"
  onkeydown="if(event.key==='Enter') crearUsuario()"
  oninput="this.classList.remove('error')" />
```

5. Mejora 4 - Filtro de busqueda

Se agregaron dos campos sobre la lista de tareas para filtrar en tiempo real. Uno filtra por ID exacto y otro por nombre (busqueda parcial). No se hace ninguna peticion adicional al servidor; el filtrado ocurre sobre los datos ya cargados en memoria.

HTML de los filtros

```
<div class="search-group">
  <input id="searchId" type="number" placeholder="Buscar por ID..."
    oninput="filtrarTareas()" />
  <input id="searchName" placeholder="Buscar por nombre..."
    oninput="filtrarTareas()" />
</div>
```

Logica de filtrado

La variable global allTasks guarda todas las tareas cargadas. filtrarTareas() lee los dos inputs, aplica los filtros sobre allTasks y llama a renderTareas() con el resultado. renderTareas() solo dibuja, no modifica allTasks.

```
let allTasks = [];

function filtrarTareas() {
  const idVal = document.getElementById('searchId').value.trim();
  const nameVal = document.getElementById('searchName').value.trim().toLowerCase();

  const filtered = allTasks.filter(t => {
    const matchId = idVal === '' || String(t.id) === idVal;
    const matchName = nameVal === '' || t.title.toLowerCase().includes(nameVal);
    return matchId && matchName;
  });

  renderTareas(filtered);
}
```

Mensaje cuando no hay resultados

renderTareas distingue entre dos casos de lista vacia: si allTasks esta vacio es porque no hay tareas en la base de datos; si allTasks tiene datos pero filtered esta vacio es porque el filtro no encontro coincidencias.

```
if (!tasks.length) {
  const emptyText = allTasks.length ? 'Sin resultados' : 'Sin tareas aun';
  lista.innerHTML = `<li class='empty-msg'>${emptyText}</li>`;
  return;
}
```

6. Mejora 5 - Edicion inline de tareas

Cada tarea tiene un boton Editar. Al hacer clic, el elemento de la lista se convierte en un formulario inline con un input para el titulo y un select para cambiar el usuario. Se puede guardar o cancelar.

Nuevo endpoint en el backend

Se agrego PUT /tasks/:id en routes/tasks.js. Recibe el nuevo titulo y el nuevo user_id, ejecuta un UPDATE en SQLite y devuelve los datos actualizados. Usa parametros preparados (?) para evitar SQL injection.

```
router.put('/:id', (req, res) => {
  const { title, user_id } = req.body;
```

```

db.run(
  'UPDATE tasks SET title = ?, user_id = ? WHERE id = ?',
  [title, user_id, req.params.id],
  function(err) {
    if (err) return res.status(400).send(err);
    res.send({ id: req.params.id, title, user_id });
  }
);
});

```

También se actualizó el GET /tasks para que devuelva el campo user_id además de title y name. Esto es necesario para que el formulario de edición pueda pre-seleccionar el usuario correcto en el dropdown.

Datos en atributos data-*

Al renderizar cada tarea, el título y el user_id se guardan en atributos data-title y data-userid del elemento li. Cuando se abre el formulario de edición, se leen desde ahí. Esto es más seguro que pasarlos directamente en un onclick con comillas.

```

<li id="task-${t.id}"
    data-title="${escapeHtml(t.title)}"
    data-userid="${t.user_id}">
  ...
  <button onclick="mostrarEdicion(${t.id})">Editar</button>
</li>

```

Función mostrarEdicion

Lee los datos del li, construye un formulario HTML y reemplaza el contenido del li con ese formulario. El select del formulario ya tiene pre-seleccionado al usuario actual de la tarea.

```

function mostrarEdicion(id) {
  const li = document.getElementById(`task-${id}`);
  const currentTitle = li.dataset.title;
  const currentUserId = parseInt(li.dataset.userid);

  const optionsHtml = allUsers.map(u =>
    `<option value="${u.id}" ${u.id === currentUserId ? 'selected' : ''}>
      #${u.id} - ${escapeHtml(u.name)}
    </option>`
  ).join('');

  li.innerHTML = `
    <div class='edit-form'>
      <input id='edit-title-${id}' value='${escapeHtml(currentTitle)}' />
      <select id='edit-user-${id}'>${optionsHtml}</select>
      <button onclick='guardarEdicion(${id})'>Guardar</button>
      <button onclick='cargarTareas()'>Cancelar</button>
    </div>
  `;
}

```

Función guardarEdicion

Lee los valores del formulario inline, los valida igual que al crear, hace el fetch PUT y recarga la lista. Al recargar, el li vuelve a su estado de solo lectura.

```

async function guardarEdicion(id) {
  const title = document.getElementById(`edit-title-${id}`).value.trim();
  const user_id = document.getElementById(`edit-user-${id}`).value;

```

```
// validacion igual que crearTarea()
let valid = true;
if (!title) { ... valid = false; }
if (!user_id) { ... valid = false; }
if (!valid) return;

await fetch(`${API}/tasks/${id}`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title, user_id })
});

await cargarTareas();
}
```

7. Seguridad

Prevencion de XSS

Cualquier texto proveniente del servidor que se inserta en innerHTML pasa por la funcion escapeHtml(). Esto convierte caracteres especiales como < > & en sus equivalentes HTML seguros, evitando que un nombre de usuario o tarea con codigo malicioso se ejecute en el navegador.

```
function escapeHtml(str) {  
  return String(str)  
    .replace(/&/g, '&amp;')  
    .replace(/</g, '&lt;')  
    .replace(/>/g, '&gt;')  
    .replace(/"/g, '&quot;');  
}
```

Prevencion de SQL injection

Todos los queries de SQLite usan el caracter ? como marcador de posicion. El driver reemplaza esos marcadores con los valores reales de forma segura, sin concatenar strings. Nunca se construye una consulta SQL mezclando texto del usuario.

```
// Correcto  
db.run('INSERT INTO users(name) VALUES(?)', [name], cb);  
  
// Nunca hacer esto  
db.run(`INSERT INTO users(name) VALUES("${name}")`, cb);
```

8. Resumen de cambios por archivo

routes/tasks.js:

- GET /tasks ahora incluye user_id en la respuesta.
- Se agrego PUT /tasks/:id para editar titulo y usuario de una tarea.

public/index.html:

- El input de ID de usuario fue reemplazado por un select.
- Se agregaron dos inputs de busqueda sobre la lista de tareas.
- Se agregaron atributos onkeydown (Enter para enviar) y oninput (quitar error) a los inputs.

public/styles.css:

- Archivo reescrito desde cero.
- Reset, grid de dos columnas responsive, tarjetas, jerarquia de botones, estados de error, formulario de edicion inline.

public/app.js:

- Variables globales allTasks y allUsers para manejar estado en memoria.
- Funcion escapeHtml() para prevenir XSS.
- Validacion en crearUsuario(), crearTarea() y guardarEdicion().
- Funcion filtrarTareas() para busqueda en tiempo real.
- Funciones mostrarEdicion() y guardarEdicion() para edicion inline.